



Software Engineering

Smart Traffic System using K Framework

Group: 30431/1

Author: Asaftei Alexandru-Petru

Teaching Assistant: Eneia Todoran





Abstract

The **Verified Smart Traffic System** is a distributed traffic simulation platform designed to guarantee operational safety in critical intersection control. The system integrates a **Django-based** web interface for visual monitoring and control, with the **K Framework** serving as the core engine for formal verification.

The primary innovation of this project is the implementation of **Runtime Verification**. Unlike traditional systems that rely solely on internal logic, this system delegates critical state-transition decisions to a formal mathematical model. Before any traffic light changes color in the simulation (e.g., Green to Yellow), the proposed transition is intercepted and verified against a set of formal rewrite rules defined in K.

This architecture ensures that the system never enters an undefined or unsafe state (such as conflicting green lights), demonstrating how formal methods can be effectively integrated into modern web applications to enhance reliability in safety-critical domains.





Contents

1. Introduction.....	4
1.1 Technologies Used	5
1.2 Objectives.....	5
2. System Architecture & Diagrams	5
2.1 Use Case Diagram	5
2.2 Deployment Diagram	5
2.3 Sequence Diagram (Verification Flow).....	6
2.4 Class Diagram	6
3. Formal Specification (K Framework)	Error! Bookmark not defined.
3.1. Syntax and Configuration	9
3.2. Safety Rules (Semantics).....	9
4. Runtime Safety Monitor	11
4.1 The Interceptor Pattern	11
4.2 Handling Safety Violations	11
5. Implementation	12
5.1. traffic.k (Formal Definition).....	12
5.2. verification.py (The Bridge)	13
5.3. scheduler.py (Simulation Logic)	15
6. Conclusion	17
7. References	17
8. Mini Project	17





1. Introduction

Traffic control systems are classified as **safety-critical systems**, where software failure can lead to severe physical consequences. Traditional software engineering relies heavily on unit and integration testing; however, testing can only show the presence of bugs, not their absence.

Safe Traffic System addresses this challenge by implementing **Formal Verification**. Instead of relying purely on Python code to manage the state machine of the traffic lights, the system uses an external, mathematically proven model to validate every decision.

1.1. Technologies Used

The project is built using a microservices-inspired architecture:

1. **Django (Python):** Serves as the backend, managing the database, the API for the dashboard, and the scheduling logic.
2. **K Framework:** A semantics framework used to define the rigorous rules of the traffic lights. It acts as the "Supreme Court" for the system's logic.
3. **APScheduler:** Handles the time-based simulation events (ticks).
4. **JavaScript (AJAX):** updates the frontend dashboard in real-time without page reloads.

1.2. Objectives

The main objective is to demonstrate a "Correct-by-Construction" approach where:

- The Python code proposes a state change.
- The K Framework verifies the safety of the change.
- The execution happens only if the verification returns a SAFE token.

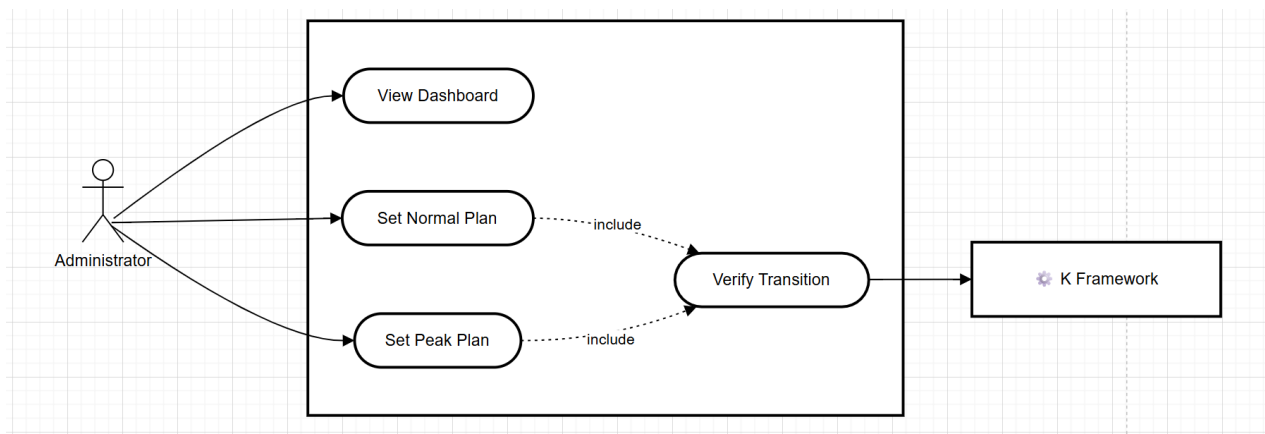




2. System Architecture & Diagrams

2.1 Use Case Diagram

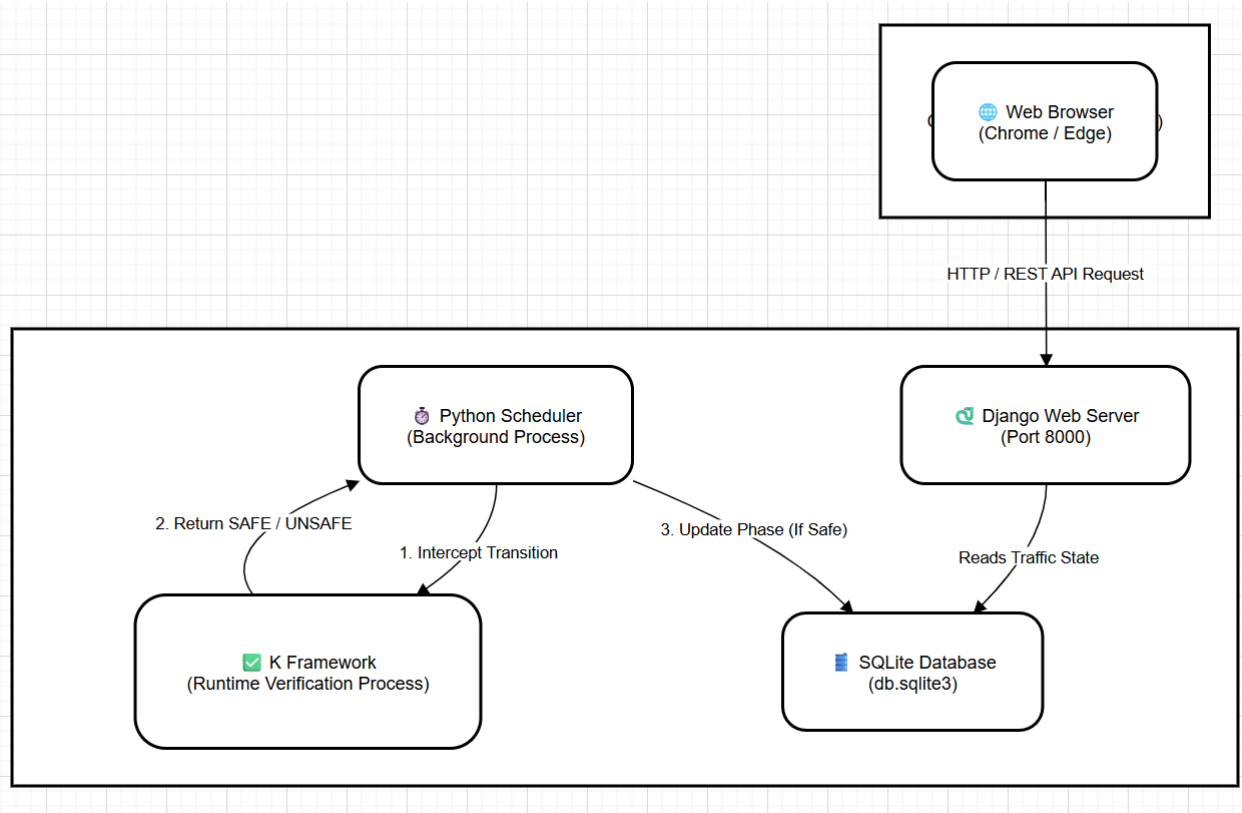
This diagram illustrates the interaction between the user (Administrator) and the system. The administrator can monitor the dashboard and select specific traffic plans (Normal or Peak). The distinctive feature of this design is the <<include>> relationship, which demonstrates that any traffic change action automatically triggers the mandatory validation process via the external 'K Framework' actor. This ensures that no human decision can bypass the safety verification layer.



2.2 Deployment Diagram

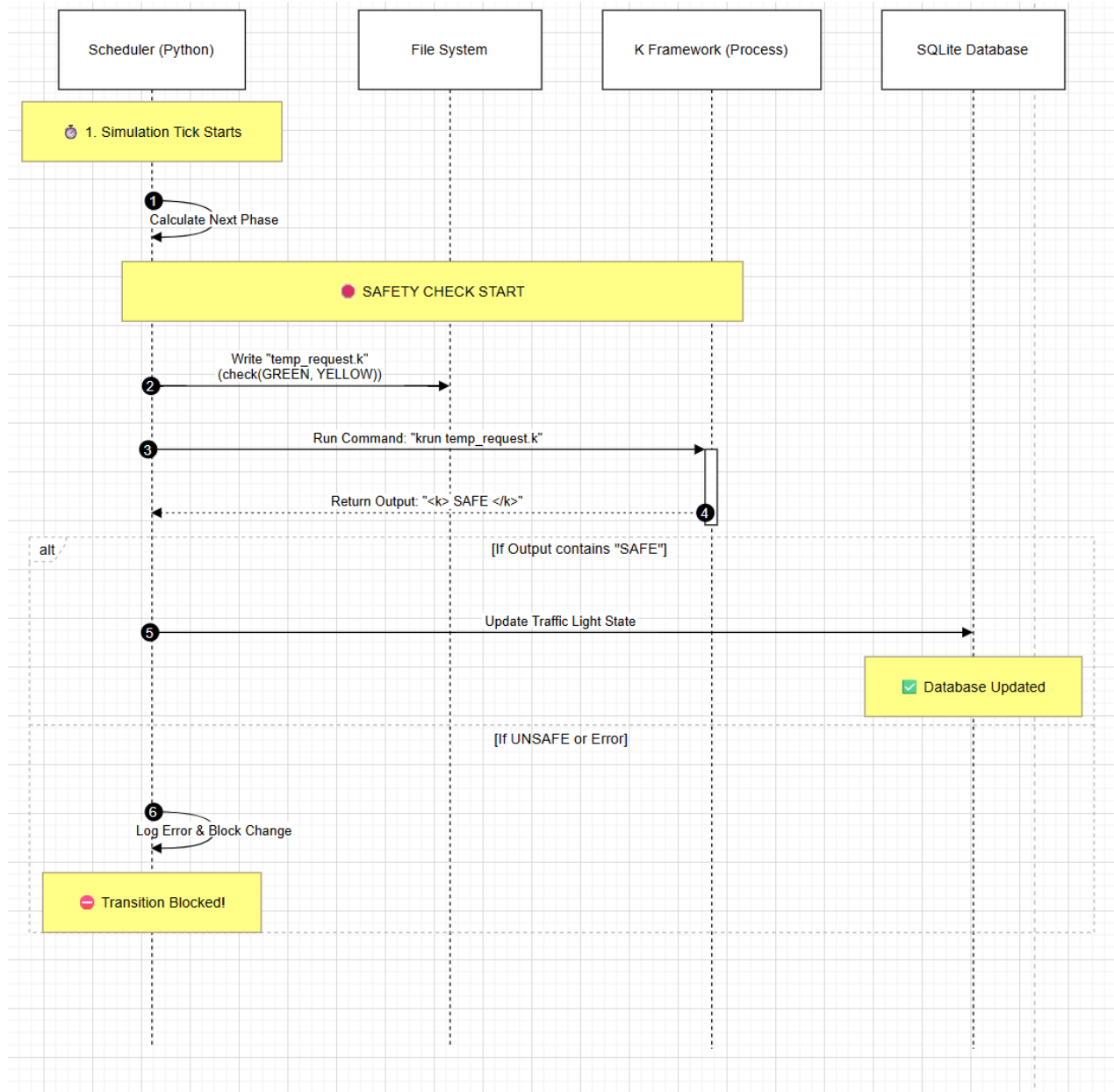
The deployment diagram presents the physical and logical architecture of the project. The system is distributed between the Client Workstation (Windows/Browser) and the Server Environment (WSL/Ubuntu). The key components highlighted are the 'Python Scheduler' and 'K Framework', which intercept the data flow towards the SQLite database. The numbered arrows demonstrate the protection mechanism: the database is updated only after the K Framework confirms the transition is safe.





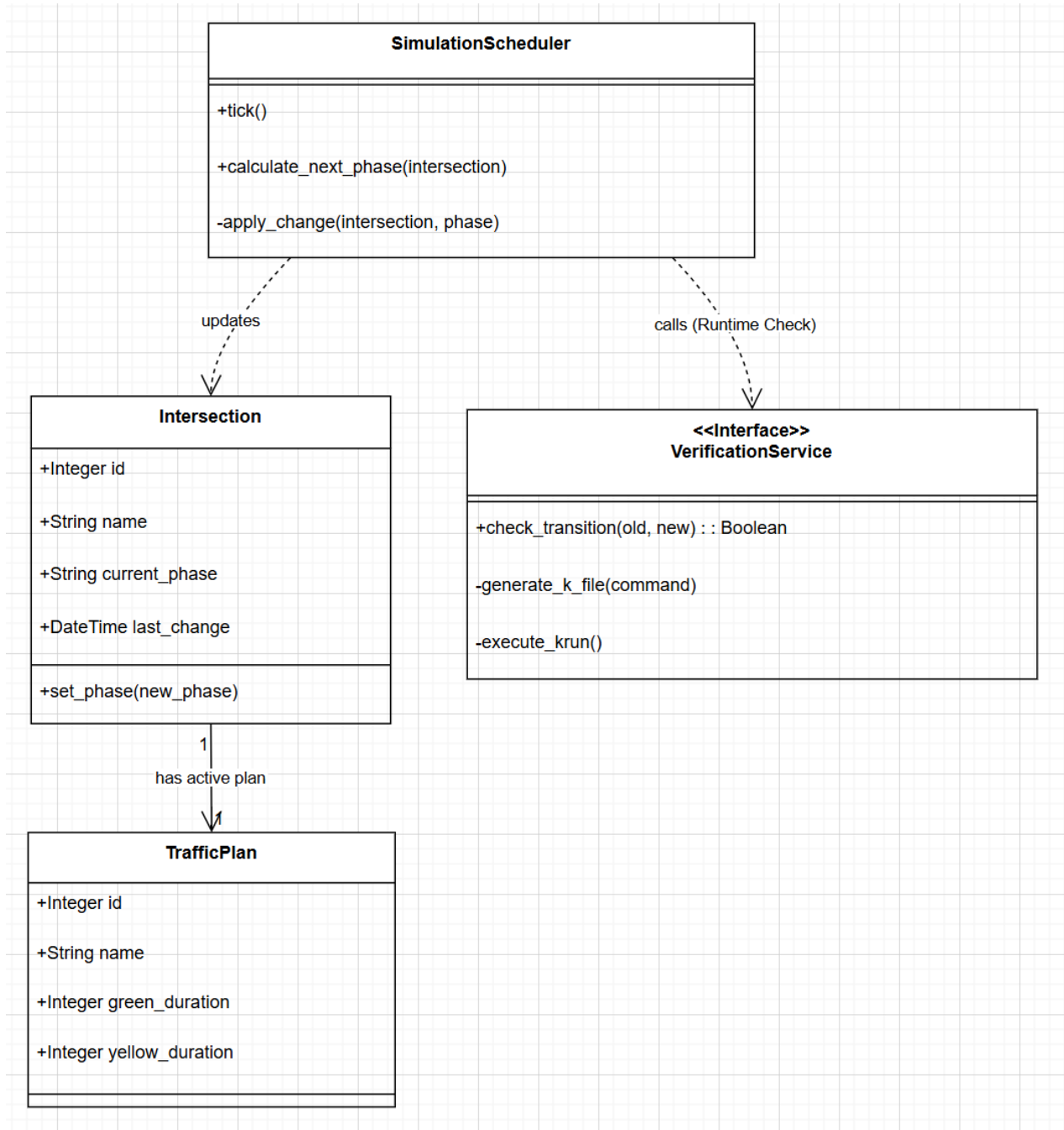
2.3 Sequence Diagram

The sequence diagram details the exact timeline of a single simulation 'tick'. It highlights the strict verification protocol: (1) The Scheduler calculates the next phase, (2) writes a temporary .k file, (3) invokes the external krun process, and (4) waits for the verdict. The alt block demonstrates the 'Fail-Safe' behavior: if the response does not contain the SAFE token, the transaction is immediately blocked, preventing the database from being updated with an invalid state.





2.4. Class Diagram





The Class Diagram provides a structural overview of the system's backend. It highlights the core Django models, `Intersection` and `TrafficPlan`, which store the state and configuration of the traffic network.

The logic layer is separated into two key components:

1. **SimulationScheduler:** Acts as the controller that manages time and initiates state changes based on the active traffic plan.
2. **VerificationService:** A specialized utility class that acts as a bridge to the K Framework. It is responsible for generating the formal definition files and executing the `krun` process. The dependency arrow shows that the Scheduler cannot update an `Intersection` without first invoking the `VerificationService`.

3. Formal Specification (K Framework)

Unlike standard programming languages, **K Framework** allows us to define the **Operational Semantics** of the traffic system. We treat the traffic light configuration not as variables, but as mathematical terms.

3.1. Syntax and Configuration

We define a Domain Specific Language (DSL) for our traffic system.

- **Colors:** Defined as syntax tokens (`NS_GREEN`, `NS_YELLOW`, etc.).
- **Result:** The output of a computation can only be `SAFE` or `UNSAFE`.
- **Command:** A function signature `check(Color, Color)` that the Python system will invoke.





```
// 1. Define Colors (Syntax)
syntax Color ::= "NS_GREEN" | "NS_YELLOW" | "EW_GREEN" | "EW_YELLOW" | "ALL_RED"

// 2. Define the Result (SAFE or UNSAFE)
syntax Result ::= "SAFE" | "UNSAFE"

// 3. Define the verification command
syntax Command ::= "check" "(" Color "," Color ")"

// 4. Program Configuration
configuration <k> $PGM:Command </k>
```

3.2. Safety Rules (Semantics)

The logic is defined via **Rewrite Rules**. A rule tells the K machine: *"If you see this pattern, replace it with that result."*

If a transition is attempted that does not match any rule (e.g., jumping from Green directly to Red, skipping Yellow), the K machine will get stuck or not return SAFE. This implicitly marks the transition as dangerous.

Defined Rules:

1. **Green to Yellow:** Permitted for both North-South and East-West directions.
2. **Yellow to All Red:** Permitted (Safe stop).
3. **All Red to Green:** Permitted (Safe start).





```
// Green -> Yellow is allowed (SAFE)
rule check(NS_GREEN, NS_YELLOW) => SAFE
rule check(EW_GREEN, EW_YELLOW) => SAFE

// Yellow -> Red is allowed (SAFE)
rule check(NS_YELLOW, ALL_RED) => SAFE
rule check(EW_YELLOW, ALL_RED) => SAFE

// Red -> Green is allowed (SAFE)
rule check(ALL_RED, NS_GREEN) => SAFE
rule check(ALL_RED, EW_GREEN) => SAFE
```

4. Runtime Verification Monitor

The project implements a **Runtime Monitor** using the Python Decorator pattern. This ensures that the verification logic is decoupled from the business logic.

4.1 The Interceptor Pattern

The file `verification.py` contains a function wrapper `@ensure_safe_transition`. Whenever the scheduler attempts to modify an intersection's phase:

1. The wrapper intercepts the call.
2. It identifies the `current_phase` and the proposed `new_phase`.
3. It constructs a K-program string dynamically: `check(NS_GREEN, NS_YELLOW)`.
4. It invokes the K compiler/runner externally.

4.2 Handling Safety Violations

The system is designed to be **Fail-Safe**. If the K Framework output does not strictly contain the token `SAFE`:

- The transition is immediately aborted.
- The database write operation is cancelled.
- A critical error is logged in the system console:  [K-Framework] UNSAFE! Transition blocked.





This guarantees that even if the Python scheduler has a bug (e.g., a logic error that tries to skip the Yellow phase to save time), the Formal Verification layer will prevent the unsafe state from reaching the physical world (or the simulation dashboard).

5. Implementation Details

5.1. traffic.k (Formal Definition)

```
module TRAFFIC
```

```
// 1. Define Colors (Syntax)
```

```
syntax Color ::= "NS_GREEN" | "NS_YELLOW" | "EW_GREEN" | "EW_YELLOW" | "ALL_RED"
```

```
// 2. Define the Result (SAFE or UNSAFE)
```

```
syntax Result ::= "SAFE" | "UNSAFE"
```

```
// 3. Define the verification command
```

```
syntax Command ::= "check" "(" Color "," Color ")"
```

```
// 4. Program Configuration
```

```
configuration <k> $PGM:Command </k>
```

```
// --- SAFETY RULES ---
```

```
// Green -> Yellow is allowed (SAFE)
```

```
rule check(NS_GREEN, NS_YELLOW) => SAFE
```

```
rule check(EW_GREEN, EW_YELLOW) => SAFE
```





```
// Yellow -> Red is allowed (SAFE)
rule check(NS_YELLOW, ALL_RED) => SAFE
rule check(EW_YELLOW, ALL_RED) => SAFE

// Red -> Green is allowed (SAFE)
rule check(ALL_RED, NS_GREEN) => SAFE
rule check(ALL_RED, EW_GREEN) => SAFE

// Any other combination (e.g., Green -> Red directly) has no rule, so it will not yield SAFE.
Endmodule
```

5.2. verification.py (The Bridge)

```
import subprocess
import os

def check_with_k_framework(old_phase, new_phase):
    """
    This function writes a small K program, runs it via 'krun',
    and checks if the result is SAFE.
    """
    # 1. Write the command to a temporary file (e.g., check(NS_GREEN, NS_YELLOW))
    # This creates the file that K Framework will read
    k_code = f"check({old_phase}, {new_phase})"

    with open("temp_request.k", "w") as f:
```





```
f.write(k_code)

try:
    # 2. Run krun
    # --term ensures we get the clean result
    # This is the command that executes formal verification
    output = subprocess.check_output(
        ["krun", "temp_request.k"],
        stderr=subprocess.STDOUT
    ).decode("utf-8")

    # 3. Check if K Framework responded with <k> SAFE </k>
    # The K output looks something like this: <k> SAFE </k>
    if "SAFE" in output:
        return True
    else:
        # If K returns something else (e.g., gets stuck), it means UNSAFE
        print(f"[K-FRAMEWORK] REJECTED! Transaction {old_phase} -> {new_phase} is not Safe.")
        return False

except Exception as e:
    print(f"Error executing K: {e}")
    return False

def ensure_safe_transition(func):
```





```
"""
The decorator that protects the traffic light change function.
"""

def wrapper(intersection, new_phase):
    old_phase = intersection.current_phase

    # --- VERIFICATION TAKES PLACE HERE ---

    print(f"🔍 [K-Framework] Verifying the transaction: {old_phase} -> {new_phase}...")

    is_safe = check_with_k_framework(old_phase, new_phase)

    if is_safe:
        # If K says YES, we execute the switch
        print("✅ [K-Framework] SAFE. Execute the switch.")
        return func(intersection, new_phase)
    else:
        # If K says NO, we block everything. The traffic light will not change.
        print("🚫 [K-Framework] UNSAFE! Transaction blocked.")
        return None

return wrapper
```

5.3. scheduler.py (Simulation Logic)

```
from datetime import datetime
from django.utils import timezone
```





```
from .models import Intersection
from .services import set_intersection_phase

def simulation_tick_job():
    # Retrieve ALL intersections from the database
    intersections = Intersection.objects.all()

    for intersection in intersections:
        if not intersection.active_plan:
            continue # Skip if no active plan assigned

        plan = intersection.active_plan
        now = timezone.now()

        # Calculate how long the light has been in the current state
        time_in_phase = (now - intersection.last_phase_change).total_seconds()

        current_phase = intersection.current_phase
        next_phase = current_phase

        # --- TRANSITION LOGIC (State Machine) ---
        if current_phase == 'NS_GREEN':
            if time_in_phase >= plan.green_duration:
                next_phase = 'NS_YELLOW'
        elif current_phase == 'NS_YELLOW':
            if time_in_phase >= plan.yellow_duration:
                next_phase = 'ALL_RED'
        elif current_phase == 'EW_GREEN':
            if time_in_phase >= plan.green_duration:
                next_phase = 'EW_YELLOW'
        elif current_phase == 'EW_YELLOW':
            if time_in_phase >= plan.yellow_duration:
                next_phase = 'ALL_RED'
        elif current_phase == 'ALL_RED':
            if time_in_phase >= 2: # Minimal safety buffer
                if intersection.previous_phase == 'NS_YELLOW':
                    next_phase = 'EW_GREEN'
            else:
```





```
next_phase = 'NS_GREEN'
```

```
# If a change is needed, trigger the verified transition
if next_phase != current_phase:
    # This call is intercepted by K Framework verification
    set_intersection_phase(intersection, next_phase)
```

6. Conclusion

The **Verified Smart Traffic System** successfully demonstrates the integration of formal methods into a practical web application. By using **K Framework**, we established a rigorous safety layer that operates independently of the main application logic.

The results show that the system correctly validates safe transitions and effectively blocks unsafe ones during runtime. This architecture is highly scalable and provides a blueprint for future safety-critical systems where the flexibility of Python and the mathematical certainty of Formal Methods are both required.

7. References

1. **K Framework Documentation:** <https://kframework.org/>
2. **Django Web Framework:** <https://www.djangoproject.com/>
3. **Runtime Verification Inc:** <https://runtimeverification.com/blog/>
4. **Formal Methods in Safety-Critical Systems:** *Introduction to Formal Methods*, J. Woodcock et al.





Hello World Mini Project

1. core/models.py

```
from django.db import models

class Message(models.Model):
    text = models.CharField(max_length=100)

    def __str__(self):
        return self.text
```

2. core/views.py

```
from django.shortcuts import render

from .models import Message

def hello(request):

    message = Message.objects.first()

    return render(request, 'core/hello.html', {'message': message})
```

3. core/urls.py

```
from django.urls import path

from .views import hello

urlpatterns = [ path('', hello, name='hello'), ]
```

4. core/templates/hello.html

```
<!DOCTYPE html>

<html lang="en">
```





```
<head>

  <meta charset="UTF-8">

  <title>Hello</title>

</head>

<body>

  <h1>{{ message.text }}</h1>

</body>

</html>
```

